Online Resource 1

Protocols and instruments

Supplementary material to: *How the Classical Baseline Shapes Machine-Learning Speed Claims in Topology Optimization.* Shaoliang Yang and Jun Wang, Santa Clara University. Corresponding author: Jun Wang, jwang22@scu.edu. Results in Engineering.

Reading guide. The documents below preserve the written plans and their dated amendments. Plans for human verification were superseded by model coding and adjudication; the realized procedure is described in Sect. 3.6 of the paper and the audit deviation log. Abbreviations: TO, topology optimization; ML, machine learning; AI, artificial intelligence; LLM, large language model; FE/FEM, finite-element analysis/method; CNN, convolutional neural network; GAN, generative adversarial network; VAE, variational autoencoder; PDE, partial differential equation; CPU/GPU, central/graphics processing unit; PCG/CG, preconditioned/conjugate gradient; SIMP, solid isotropic material with penalization; OC, optimality criteria; MBB, Messerschmitt–Bölkow–Blohm beam; DOF, degree of freedom; RAM, random-access memory; GiB, gibibytes; CI, confidence interval. OAT is Optimize Any Topology, a diffusion model; HGTO is the graph-based design method. Gate labels G1 and G2 denote the audit-framing and baseline-correctness checks defined below. Exact prompts, search queries and the coding schema are quoted instruments; their machine-readable keys are retained. In these documents, pre-registered means fixed in writing before the coding or runs it governs, in a hashed plan or a dated log row entered before the run; the hashes are held in the authors' records, without an external registry timestamp. Further abbreviations: CE, compliance error; DDIM, denoising diffusion implicit model; RAMP, rational approximation of material properties; ESO/BESO, (bi-directional) evolutionary structural optimization; MSE, mean squared error; IoU, intersection over union; MKL, Intel Math Kernel Library; CSR, compressed sparse row; UTC, coordinated universal time. PGD names a pyEDGE solver option.

This resource reproduces the protocol documents as they stand at submission. Each is hashed in the project manifest, and every change after the first coding record is listed with its date and reason in the deviation logs. The wording is unchanged except that internal working labels are replaced by plain terms (for example, first-pass coding and seeded order), the journals named in the G1 gate are replaced by their type, planned output labels are replaced by the final location of that output, a local software path and a working-folder path are replaced by descriptions, one mention of an internal review and two references to internal working notes are removed, and one administrative remark is shortened. File names and quoted instrument keys are unchanged; field names outside the quoted instruments are rendered as plain labels, and list markers are typeset as numbers. The unedited documents and their hashes are in the repository. Section numbers inside the documents are the documents' own; pointers to the paper use its current section and figure numbers.

1. Audit protocol (pre-registered, with its deviation log in §11)

Audit protocol — pre-registered

Version: 1.0 · **Written:** 2026-09-24 · **Freeze rule:** this file is hashed into `evidence/MANIFEST.sha256` before the first coding record or timing run exists. Any later change goes into §11, the deviations log, with date and reason. Nothing is edited silently.

The design follows McGreivy & Hakim (*Nat. Mach. Intell.* 2024, doi 10.1038/s42256-024-00897-5; arXiv 2407.07218). Their Methods were read on 2026-09-24. Two differences matter:

1. **They excluded solid mechanics and metamaterials.** Their exclusion list reads "solid mechanics … meta-materials, hyper-elasticity". This audit covers that excluded domain.
2. **Their break-even calculation was a recommendation (Rec. 3); here it is a rule (R3).** In TO the training data are produced by the very optimizer the model replaces.

---

1. Questions

1. **C1.** Among articles that use machine learning (ML) for topology optimization (TO) and claim to outperform a classical method, what share of primary outcomes violate at least one fairness rule (R1–R4)?

2. **C2.** How much do re-tested speed-up claims shrink against a strong classical baseline at matched design quality, on one machine?
3. **C3.** How does counting data generation and training change the break-even number of uses?
4. **C4.** How common are reporting practices that hide negative results?
5. **C5 (secondary).** How well does an LLM pre-extraction agree with the human-verified verdicts?

2. Sources and search

All searches are recorded with date, query string, endpoint and raw response in `audit/search/`. The query list is frozen in `audit/search/queries.json` before any screening.

1. **Keyword queries.**
   (a) arXiv API: title and abstract fields.
   (b) OpenAlex: the `title_and_abstract.search` filter, publication date $\geq$ 2017-01-01.
   (c) Queries cover "topology optimization" and "topology optimisation" crossed with: deep learning, neural network, machine learning, generative, GAN, diffusion, variational autoencoder, flow matching, reinforcement learning, large language model, agent, neural reparameterization, implicit neural, physics-informed, surrogate.
   (d) They also cover metamaterial, microstructure and architected material crossed with inverse design and ML (restricted to TO or homogenization comparators).
2. **Forward citations** (OpenAlex citation filter) of these anchor articles:
   (a) Sosnovik & Oseledets 2017 (neural networks for TO);
   (b) Banga et al. 2018 (3D TO with CNNs);
   (c) Nie et al. 2021 (TopologyGAN, doi 10.1115/1.4049533);
   (d) Chandrasekhar & Suresh 2021 (TOuNN);
   (e) Woldseth et al. 2022 (doi 10.1007/s00158-022-03347-1);
   (f) Mazé & Ahmed 2023 (TopoDiff, arXiv 2208.09591). Each anchor's identifier is resolved and recorded before use. An anchor that cannot be resolved is logged in §11 and dropped.
3. **Backward citations**: reference lists of Woldseth et al. 2022, Sanu et al. 2025 and OAT (arXiv 2510.23667), matched by title.

Search end date. The date the queries are frozen is recorded in `queries.json`. Only articles available on or before that date are included.

3. Inclusion criteria (all must hold)

1. **Method.** A learned model is used to produce, accelerate or replace iterative TO. Learned models include neural networks, Gaussian processes, reinforcement learning, diffusion or flow models, GANs, VAEs, transformers and LLMs.
   (a) TO covers density-based methods (SIMP, RAMP), level-set, moving morphable components/voids, and ESO/BESO.
   (b) Physics covers linear or nonlinear elasticity (compliance, stress, displacement), heat conduction, and compliant mechanisms.
   (c) Also included: inverse design of architected materials or metamaterials, where the classical comparator is TO or homogenization-based optimization.
2. **Comparison.** The article's **primary outcome** (§5.1) compares the ML method with a classical, non-learned optimization method on **speed or computational cost, or a proxy for them**. Examples: "N× faster", "real-time", "fewer FE evaluations", "without iterative optimization". This matches the template's speed-based inclusion.

(a) Articles whose primary outcome compares only design quality with a classical method are coded too. They are analysed as a **secondary stratum (Q)** and are not pooled into the headline C1 rate.

3. **Type.** Journal article, conference paper, or arXiv preprint (latest version). English.
4. **Date.** 2017-01-01 to the search end date.

4. Exclusion criteria

1. Reviews, surveys, theses, dissertations, presentations, technical reports, patents, abstracts-only records.
2. Articles with no comparison to a classical method. Articles that compare only with ML baselines are tallied as **ML-only** (reported under C4) and not coded for R1–R4.
3. ML used only for forward analysis, with no optimization-level comparison.
4. Fluid-flow TO (the template's domain overlaps it), acoustic, photonic and electromagnetic TO.
5. Full text not accessible to the coders (tallied separately; a stated limitation).
6. Duplicates. Code the version of record when accessible, otherwise the most complete preprint; record both IDs.

5. Primary outcome and fairness rules

5.1 Primary outcome (template rule, verbatim in substance)

1. If the abstract contains a quantitative comparison with a classical method, that is the primary outcome.
2. Otherwise, take the first quantitative comparison in the conclusion, then the introduction, then the main text.
3. If no quantitative comparison exists anywhere, take a qualitative one ("significantly faster", "outperforms"), in the order abstract → conclusion → introduction.
4. If there are several candidates, the coder picks the one the article emphasises most, and records the alternatives.

5.2 Scale (template convention)

1. ✓ **pass** — the rule is satisfied, **or the coder is unsure**.
2. ! **partial** — partly satisfied; or likely not satisfied without enough evidence to be sure.
3. ✖ **fail** — not satisfied.
4. **n/a** — the rule does not apply.

The headline counts ✖ only. Online Resource 4 reports the sensitivity with ! counted as ✖.

5.3 Rules R1 — matched quality or matched time. (Template rule 1, adapted to design quality.)

1. ✓ The speed comparison is made at equal design quality: same volume fraction on the physical field, and compliance or the objective within the ML method's reported error of the classical result. Or it is made at equal wall-clock or equal FE-evaluation budget.
2. ✖ Any of:
   (a) ML inference time is compared with a classical run to full convergence, while the ML output is measurably worse (outside the reported error) and no matching is attempted;
   (b) a speed claim is made with no quality comparison;

(c) post-processing (e.g. SIMP refinement steps) is used for the quality result but not counted in the reported time.

3. ! Quality is reported but matching is loose; for example, the classical stopping rule is unstated.

R2 — efficient classical baseline. (Template rule 2, adapted.) The state equation in TO is elliptic. Following the template's elliptic-PDE criterion, efficient means a sparse direct solver (Cholesky/LDL^{T} class) or a multigrid-preconditioned Krylov solver, on hardware comparable to the ML method.

1. ✖ Any of:
   (a) **(a)** Our re-implementation of the paper's classical method reaches its primary-outcome result significantly faster: $\geq 3\times$ on the same hardware class at the same size (after the re-test).
   (b) **(b)** The abstract reports a weaker baseline than the results or appendix.
   (c) **(c)** A GPU ML method is compared with a CPU-only classical run on problems of $\geq 10^5$ degrees of freedom, with no justification. This is template Rec. 2, made concrete.
   (d) **(d)** The classical run uses a fixed iteration count at least $2\times$ what the article's own convergence criterion or convergence plot needs.
   (e) **(e)** The reported per-iteration time of the classical method is $\geq 10\times$ our strong baseline's at the same size on the same hardware class. Per-iteration time must be derivable from the article.
2. ! Any of:
   (a) the implementation looks inefficient (e.g. an unvectorized educational code, or a general-purpose commercial TO module), but no direct comparison is possible;
   (b) the hardware choice is not justified.

R3 — full cost accounting. (TO-specific; template Rec. 3 made a rule.)

1. ✓ A speed claim reports both data-generation cost and training cost, and either gives the break-even number of uses or explicitly limits the claim to per-query inference.
2. ! Training cost is reported but data-generation cost is missing, or costs are reported with no amortization.
3. ✖ A speed claim with no training or data cost at all.
4. n/a Per-problem learning with no offline training (e.g. neural reparameterization): its cost sits inside R1/R2.

R4 — verified outputs. (TO-specific.)

1. ✓ Quality is computed by an FE analysis of the **returned** design, including post-processing when used. Volume or constraint satisfaction is reported, and infeasible or failed designs (disconnected, floating material, volume violation) are counted.
2. ! FE-verified, but constraint violations or failure rates are not reported.
3. ✖ Quality rests only on image-similarity metrics (MSE, pixel accuracy, IoU), on surrogate-predicted performance, or on examples selected without a stated rule.

Fair = no ✖ on any applicable rule. **Weak baseline (template-comparable)** = ✖ on R1 or R2.

6. Reporting indicators (C4)

Each is recorded as yes, no or unclear, with a locator:

1. claim direction (positive, negative, mixed);
2. number of test cases;

3. out-of-distribution test present;
4. seeds or variance reported;
5. failure or infeasibility rate reported;
6. timing protocol stated (hardware, warm-up, iteration count);
7. runtimes reported for all cases or selectively;
8. code available;
9. data available;
10. hardware for each method stated.

Two field-level counts are also kept: the share of included articles claiming to outperform versus to underperform (the template's publication-bias signal), and the ML-only tally from §4.

7. Coding procedure

1. **LLM pre-extraction.** An LLM reads the accessible full text and fills the record schema (`audit/schema/coding_schema.json`), with a locator for each field (section, table, figure or page) and at most one short key phrase (< 15 words) per article.
   (a) The first-pass coding is saved to `audit/coding/llm_prefill/` and hashed **before** human verification.
   (b) Model name and date are recorded.
2. **Human verification.** The human coder checks every field against the article and sets the final verdict and their initials in `audit/coding/records/`. **A record without verifier initials does not count** in any reported rate.
3. **Second coder.** A second human codes a random 20 % subset independently. Cohen's κ is reported per rule.
4. **Copyright and storage.** No third-party PDFs are stored in the project. Records hold identifiers, URLs, locators, paraphrases and short key phrases only.

8. Gates

Gate	Measurement	$\geq$ upper	between	$<$ lower
G1	Share of the 20 pilot primary outcomes with ✖ on R1 or R2 (human-verified; speed stratum)	$\geq$ 40 % → full audit, broad-audience journal framing	$\geq$ 20 % and $<$ 40 % → full audit, specialist-journal framing	$<$ 20 % → stop the audit; publish the strong baseline as a short benchmark note, or switch to the runner-up (dataset admissibility)
G2	Strong-baseline wall-clock for a 150-iteration SIMP run at 256×256 on this machine, against OAT's reported 68.30 s (arXiv 2510.23667 Table 6, SIMP-GPU)	$\leq$ 13.66 s → re-test headline	$>$ 13.66 s and $<$ 68.30 s → moderate	$\geq$ 68.30 s → OAT's baseline was fair

1. **The pilot sample** is 20 articles drawn uniformly at random, seed **20260924**, from the included set with accessible full text, after title/abstract screening is complete.
2. **A provisional G1** may be computed from first-pass LLM codings, labelled provisional. The decision uses human-verified records only.
3. **A provisional G2** uses the pre-registered standard formulation (`plan/BASELINE_PREREG.md`). The final G2 uses OAT's own generator settings once its code may be downloaded.

9. Statistics

Method credits. Exact binomial intervals follow Clopper and Pearson (1934); the chance-adjusted agreement statistic follows Cohen (1960). The paired sign test follows Dixon and Mood (1946). These credits identify the named methods without changing the protocol.

1. Proportions with exact Clopper–Pearson 95 % intervals.
2. Cohen's κ with bootstrap 95 % intervals (2,000 resamples, seed 20260924).
3. Speed-up ratios as medians with interquartile ranges. No pooling across hardware classes.
4. No hypothesis test is used to decide a gate. Gates are threshold rules on point estimates, as stated.

10. What this audit does not claim

1. It does not claim that ML has no value for TO. Design diversity, differentiable priors, out-of-distribution exploration and access for users without solvers are outside the rules.
2. It does not judge any author's intent.
3. It does not re-test every article. Re-tests cover the articles whose code can be run and whose licences allow running it.

11. Deviations log

Date	Section	Change	Reason
2026-09-24	§2 sources	The OpenAlex queries, forward-citation and backward-citation searches run when the OpenAlex daily budget resets. The arXiv queries ran on 2026-09-24 (523 unique records). Semantic Scholar is also rate-limited (HTTP 429).	The shared free OpenAlex budget for this network was exhausted on 2026-09-24. Recorded in `audit/search/raw/requests.jsonl`
2026-09-24	§2–4 screening method (clarification before any screening)	Title/abstract screening is pre-screened by an LLM (Claude agents) against the §3–4 criteria, with a reason code per record. The human coder confirms every include and a random 10 % of excludes (seed 20260924) before full-text coding. Records without human confirmation count only in provisional numbers	Feasibility at pool sizes of hundreds to thousands. This mirrors the §7 code-then-verify design
2026-09-24	§8 pilot	While the pool is arXiv-only, a **provisional** pilot is drawn from the arXiv stratum (same seed and size). The pilot for the G1 decision is redrawn from the complete included set once all sources have run	Keeps coding moving without letting a partial pool decide G1
2026-09-24	§8 pilot size (clarification, written before any full-text coding)	G1 needs 20 *speed-stratum* primary outcomes. If fewer of the 20 drawn pilot papers end up in the speed stratum after full-text coding (excluded, or quality-only), add papers from the remaining arXiv-stratum includes in a fixed order until 20 speed-stratum outcomes exist. The order is a shuffle with `random.Random(20260925)`, stored in `audit/screening/pilot_topup_order.csv`	The 20 drawn papers mix I-SPEED, I-UNCLEAR and I-QUALITY screening codes, so some will leave the speed stratum at full text. Fixing the top-up order now keeps it independent of coding results
2026-09-24	§3.2 / §4 no-baseline articles (clarification after the first 20 first-pass LLM codings; before any human verification)	"Comparison with a classical method" means the article runs, times or cites a classical method's cost or result. A purely qualitative motivation with no classical run and no timing is **no comparison** and is excluded as `FT-NOBASELINE`. This mirrors the template, which excluded articles that compare to no baselines. Human verification applies this rule. Affected first-pass codings: P00002, P00015, P00519 (coded as includes with R1 = fail)	The LLM coders read the rule inconsistently. Applying it as written *lowers* the weak-baseline count, so the clarification works against the audit's hypothesis. It is recorded now, before any verified record exists

Date	Section	Change	Reason
2026-09-24	§4 full-text exclusion reasons (clarification before any human verification)	Full-text exclusions use one code from a closed list, first applicable in this order: FT-DUPLICATE, FT-SCOPE, FT-TYPE, FT-DOMAIN, FT-NOML, FT-FORWARD, FT-MLONLY, FT-NOBASELINE, FT-OTHER (definitions: `audit/schema/screening_codes.md`, second table). Earlier first-pass codings used free-text labels; `analysis/codes.py` maps them without editing the frozen first-pass codings (FT-ML-only → FT-MLONLY, FT-NOCOMPARISON → FT-NOBASELINE) and stops on any other label. Three later first-pass codings (P00252, P00313, P00041) are also flagged as FT-NOBASELINE candidates for the verifier (`audit/coding/verifier_flags.csv`)	The flow diagram (Fig. 2 of the paper) and the C4 tally need one count per reason. Free-text labels split the counts
2026-09-24	§2 searches run; anchor FC1 (after the searches, before any screening of their results)	OpenAlex OA1–OA3, forward citations FC1–FC6 and backward citations BC1–BC2 ran at about 20:00 UTC, anonymously: generic User-Agent, without an email contact header. The budget reset was not needed: 918 of 1,000 daily credits were free. FC1 ("Neural networks for topology optimization") first resolved to a wrong work (W2964971928, a graph-neural-network paper), because the title-search fallback took the most-cited non-exact hit. Its 342 results are quarantined in `audit/search/raw/_quarantine/` and excluded. The resolver now accepts exact titles only. `queries.json` v1.1 adds FC1's DOI, 10.1515/rnam-2019-0018 (W2757408183; confirmed by OpenAlex and Crossref). FC1 then returned 30 citing works; OpenAlex has no separate record of the arXiv preprint	A wrong anchor would have added hundreds of off-topic records. The fix changes only how the anchor is found, not what is searched
2026-09-24	§8 decision pilot (written while the new records are being pre-screened, before any of their screening results are seen)	After LLM pre-screening of the complete pool (all sources), one random order is fixed over all LLM includes: the includes sorted by pid, shuffled with `random.Random(20260924)`, stored in `audit/screening/pilot_final_order.csv`. G1 walks this order and uses the first 20 records that are, at full text, speed-stratum includes. Records that human screening excludes, full-text exclusions (including FT-DUPLICATE), quality-stratum records and records without accessible full text are skipped, not redrawn. An LLM exclude that the human turns into an include is appended in a second shuffle (seed 20260925). Already-coded records keep their frozen first-pass codings. The provisional arXiv pilot and its top-up list stay as recorded and are reported as the provisional G1	A seeded shuffle, then the first eligible records, is a uniform random draw from the eligible set. Fixing the order now, before the screening results, keeps G1 independent of coding results and robust to later human confirmation
2026-09-24	§2.3 backward citations: OAT's reference list (found while drafting Methods; before any screening of its records)	§2.3 names three backward sources, but `queries.json` v1.0–1.1 listed only two (Woldseth et al. 2022, Sanu et al. 2025); OAT (arXiv 2510.23667) was omitted in error. OpenAlex holds OAT (W7197034560) without a reference list, so its 64 references were parsed from the arXiv HTML bibliography, as §2.3 specifies ("matched by title"), and stored as titles, authors, venue and year (`audit/search/raw/backward/BC3/references.json`). `queries.json` v1.2 adds BC3. The pool gained 46 records (P02546–P02591); 17 references matched existing records. Existing pids are unchanged (checked)	Brings the search in line with the pre-registered source list
2026-09-24	§3.4 date window in the seeded order (clarification, before the draw)	The seeded order covers LLM includes that satisfy §3.4's date window (published 2017-01-01 or later). arXiv and OpenAlex dates decide deterministically; LLM includes dated earlier are listed in `audit/screening/pilot_final_date_excluded.json`. Records known only from OAT's parsed bibliography keep the screener's decision, because their parsed years are unreliable	Screening batches applied the date criterion inconsistently: one coded 67 pre-2017 records E-OTHER, others screened on content. The protocol's own criterion, applied mechanically, removes the inconsistency
2026-09-24	§3.3 record type in the seeded order (clarification, before the draw)	Also applied mechanically at the draw: §3.3 admits journal articles, conference papers and arXiv preprints, so an LLM include that is a preprint on another server (Zenodo, SSRN, Research Square and similar; pool type "preprint" without an arXiv id, arXiv DOI or arXiv venue) is not eligible. It is listed in `audit/screening/pilot_final_date_excluded.json`	One screening batch included about 16 such preprints on content; the protocol's type criterion excludes them

Date	Section	Change	Reason
2026-09-24	§7 verification (major change; the author's decision, recorded before any verification)	Human verification is replaced by independent AI verification. (1) Every coded record in the analysis sample is coded a second time, **blind to the first coding**, by a different model (Claude Sonnet 5; the first pass used the default Claude model, whose version was not recorded). (2) Scripts compare the two codings item by item (inclusion, exclusion code, stratum, R1–R4, claimed speed-up) and recheck derived arithmetic. (3) An adjudicator agent settles every disagreement from the article text, with locators. Final records mark whether the codings agreed or required adjudication and count in reported rates. (4) C5 becomes the agreement between the two independent AI codings (Cohen's κ per item); the human second coder of §7.3 is dropped. Every final record is published with its locators, so any verdict can be checked	The author judged human verification unnecessary and wants a self-contained pipeline with minimal human intervention. First-pass AI coders applied some rules inconsistently in this project, so one AI pass is never used alone: two independent codings plus adjudication replace the human check
2026-09-24	§2–4 and §8 analysis sample (the author's decision; recorded before any second-pass coding or adjudication)	No census. C1 and C4 are estimated on the records at **positions 1–200 of the seeded order**, which is a seeded random permutation of eligible includes, coded wherever a legal full text is accessible. G1 stays the first 20 speed-stratum records of the same order. The provisional arXiv pilot and its top-ups are reported only as a secondary sample	A prefix of a seeded random order is a simple random sample of the eligible includes, so the estimates stay unbiased for the accessible literature. Position 200 balances precision against cost
2026-09-24	§3.1 and §4 physics scope: elastic waves (clarification before any verification)	Elastic-wave, phononic and seismic metamaterial design in solids is elasticity (elastodynamics) and in scope. "Acoustic" in §4 means sound in fluids. Every screening batch already applied this reading, so no record changes	Consistent with the elasticity scope of §3.1 and with the existing screening
2026-09-24	§7 access routes and conflicts of interest (the author's decisions)	An open-access copy served from the publisher's own file host (for example mdpi-res.com, or a publisher PDF server linked from the article page) counts as accessible. No CAPTCHA or bot check is ever bypassed. Conflicts of interest confirmed by the author: P00428 and P00501 are co-authored by the author's PhD advisor, and P00497 is the author's own paper. They are coded like all other records and disclosed in the manuscript	Recorded as instructed
2026-09-24	§2–4 screening check (follows the verification decision above)	The human confirmation of title/abstract screening is replaced by AI. (1) For records in the analysis sample, full-text coding (first pass, blind second pass, adjudication) decides inclusion, as the protocol already makes the full text the arbiter. (2) The random 10 % sample of LLM excludes (seed 20260924, `audit/screening/human_check.csv`, 188 records) is re-screened blind by a different model, and the disagreement rate is reported as the estimated false-exclusion rate	Same author decision: self-contained, with minimal human intervention
2026-09-24	§5.3 rule applicability in the secondary stratum Q (clarification before any adjudication; both blind passes coded without it)	When the primary outcome compares design quality only (stratum Q): R3 is n/a, because it concerns speed claims; R2 applies only through criterion (b), a weaker baseline in the abstract than in the results or appendix, and is otherwise n/a, because its other criteria concern the cost of the classical run; R1 and R4 apply as written. Adjudicators apply this reading	The two blind passes read R2's applicability to quality claims differently (n/a against pass). Stratum Q is secondary and outside the headline rate, so the reading cannot move G1 or the headline C1
2026-09-24	§7 full-text coding without full text (clarification during the second pass)	An article whose full text no coder can read legally keeps its screening decision (include, "I-UNCLEAR (no accessible full text)", stratum none, all verdicts n/a); full-text exclusion codes need the full text. Sent to the running second-pass agents and included in later prompts; adjudicators apply it	One second-pass coder excluded an article from its abstract. Coding instructions step 2 presupposes the full text
2026-09-24	FT-DUPLICATE tie-break for identical records (clarification before any adjudication)	When two pool records hold the same version of one article (same arXiv identifier or DOI, or the same version under two records), the record earlier in the seeded order (`audit/screening/pilot_final_order.csv`; a record outside the order counts as later) carries the coding and the other is FT-DUPLICATE. Records of different versions keep the primary-version rule of `screening_codes.md`	The primary-version rule cannot choose between two records of the same version (for example P00676 and P00133, P01468 and P00141), and the coders chose differently

Date	Section	Change	Reason
2026-09-25	§7 speed-stratum author review (after adjudication; Springer Nature AI policy)	The authors scored all 112 speed-stratum verdicts from the locators. Seven differ from the adjudicated coding (P00181 R1, P00647 R1 and P00827 R1 partial→fail; P00178 R2 and P01974 R2 partial→pass; P00149 R4 and P00786 R4 partial→fail). Human–adjudicated κ is 0.83, 0.85, 1.00 and 0.88 for R1–R4. The weak share would be 11/28 under the human scores and stays 8/28 in the reported rates, which remain the adjudicated records. Log: `evidence/human_review.json`	The adjudicator used the first coder's model. The review is the human check Sect. 3.6 of the paper states. Adjudicated records are not rewritten
2026-10-28	§8 G1 framing (post hoc venue change, decided after G1)	The G1 band 20–40 % names a specialist-journal framing. The manuscript is submitted to *Results in Engineering*, which covers all areas of engineering. The framing stays that of a specialist topology-optimization paper; claims and analyses are unchanged	The journal accepts papers that are "scientifically sound, technically correct and provide valuable new knowledge to the engineering community". The change of venue does not strengthen the framing

2. Baseline pre-registration (with its deviation log in §9)

Strong classical baseline — pre-registration

Version: 1.0 · **Written:** 2026-09-24 · Frozen in `evidence/MANIFEST.sha256` before the first timing run. Deviations go in §9.

Purpose. Supply the "efficient classical method" that rule R2 and the re-tests (C2, C3) need.

1. The baseline changes **how** the state equation is solved.
2. It does **not** change **what** is optimized: formulation, filter, penalty and update stay those of the target article (or of top88 when no target applies).

This separates implementation efficiency from algorithmic tuning.

---

1. Formulation (standard modified SIMP, 2D plane stress)

1. **Mesh.** nelx × nely unit-square 4-node bilinear elements, 2×2 Gauss quadrature.
2. **Material.** $E_0 = 1$, E_min = 1e-9, $\nu = 0.3$, $E(\rho) = E_min + \rho^p(E_0 - E_min)$, $p = 3$ constant. No continuation unless a target's generator uses it.
3. **Filter.** Density filter with cone weights $w = \max(0, r_min - d)$, normalised by the row sum; this is top88 `ft = 2`. The CPU path uses a sparse H. The GPU path uses the same weights as a convolution. The two are mathematically identical.
4. **Update.** Optimality criteria: move 0.2, damping 0.5. Bisection on the physical-volume constraint until $(l_2 - l_1)/(l_1 + l_2) < 1e\text{-}3$.
5. **Initial design and objective.** x = volfrac everywhere. Compliance $c = f^T u$.
6. **Sensitivities.** $dc = -p\ \rho^{p-1}\ (E_0 - E_min)\ u_e^T\ K_e\ u_e$; dv = 1. Both are filtered by H^T, as in top88.

2. Problems for the provisional G2

ID	Domain	Supports	Load	volfrac	r_min
P1	cantilever, n × n	left edge clamped	unit downward force at the right-edge mid-node	0.4	1.5 × n/64
P2	MBB half-beam, n × n	left edge u_x = 0; bottom-right node u_y = 0	unit downward force at the top-left node	0.5	1.5 × n/64

n ∈ {64, 256}. r_min scales with n, which keeps the physical length scale constant (1.5 at 64, 6.0 at 256).

The final G2 and all re-tests replace P1 and P2 with the target article's own problem definitions and generator settings, recorded in §9.

3. Solvers

ID	Role	Method	Environment
S-GPU	strong	Matrix-free Galerkin geometric-multigrid–preconditioned conjugate gradient (PCG), float64, on the GPU	conda `torch5090`: Python 3.12.13, torch 2.11.0+cu128, CUDA 12.8, RTX 5090 (sm_120)
S-PARDISO	strong CPU	Sparse assembly in scipy CSR, reduced to free DOFs, solved by MKL PARDISO through `pypardiso`	conda `Pytorch`: Python 3.10.18, scipy 1.15.3, pypardiso 0.4.7
S-SuperLU	reference ("typical Python port")	Same assembly, `scipy.sparse.linalg.spsolve` (SuperLU)	conda `torch5090`: scipy 1.17.1

S-GPU details.

1. **Fine operator.** Matrix-free: gather u_e, apply E_e K_e, scatter-add.
2. **Levels.** Coarsen by 2 while both element counts are even and the smaller exceeds 8. That gives 256 → … → 8 (6 levels) and 64 → … → 8 (4 levels).
3. **Coarse operators.** Element-local Galerkin: coarse K_c = Σ over the 4 children of $\tilde{P}$_k^{T} K_child $\tilde{P}$_k.
   (a) P_k is the bilinear interpolation from the parent element's 8 DOFs to child k's 8 DOFs.
   (b) $\tilde{P}$_k masks fixed DOFs on both sides. Coarse fixed DOFs are coarse nodes coincident with fixed fine nodes.
   (c) This equals the global Galerkin product $\tilde{P}^{\mathrm{T}}\mathrm{K}\tilde{P}$. It is recomputed every design iteration and is included in the timing.
4. **Smoother.** Damped Jacobi.
   (a) Start with $\omega = 0.6$, V(2,2).
   (b) **Selection rule, fixed now:** if any PCG solve on P1 at n = 64 (iterations 1–150) does not reach tolerance within 200 PCG iterations, try V(3,3) with $\omega = 0.5$. The first configuration that passes is frozen for every problem and recorded in §9.
5. **Coarsest level.** Assembled dense matrix with identity rows for fixed DOFs, solved by Cholesky.
6. **Convergence.** Stop when $\|\mathrm{r_k}\|/\|\mathrm{f}\| \le$ 1e-8 by the CG recurrence. Then recompute the **true** residual $\|\mathrm{f} - \mathrm{Ku}\|/\|\mathrm{f}\|$. Accept only if it is ≤ 1e-7; otherwise keep iterating, up to 500. Every solve's true residual is logged.
7. **Warm start** from the previous design iteration's u.

Equivalence requirement. On P1 and P2 at n = 64 and n = 256, after 150 iterations, S-GPU, S-PARDISO and S-SuperLU must agree: relative difference in final compliance ≤ 1e-5, and max $|\Delta\rho_\mathrm{phys}| \le$ 1e-3. If they disagree, stop and diagnose before any timing is reported.

4. Timing protocol

1. **Fixed 150 design iterations.** This matches OAT's Table 6 protocol ("150 steps of optimization (average needed to converge)"). The first iteration at which $\|\Delta\mathrm{x}\|\infty < 0.01$ is also recorded, for information.
2. **Warm-up.** One 10-iteration run per solver and problem size, discarded.

3. **Repeats.** Three timed repeats; report the median, minimum and maximum.
4. **Timed region.** The whole design loop: filter, assembly (CPU paths), coarse-operator build (S-GPU), solve, sensitivities, OC, compliance.
   (a) Excluded: one-time setup (index arrays, K_e, hierarchy index maps). Setup is reported separately.
   (b) GPU timers synchronize CUDA execution.
5. **Per-iteration log.** Compliance, physical volume, change, solve time, PCG iterations, true residual.

5. Energy protocol

1. `nvidia-smi --query-gpu=timestamp,power.draw,clocks.sm,utilization.gpu --format=csv -lms 100` runs during the timed region.
2. A 5 s idle trace is taken first.
3. Report both GPU board energy (trapezoidal integral) and energy above idle.
4. CPU package energy is **not** measured on this Windows machine; this is a stated limitation.

6. Hardware and software record

`tools/envcapture.py` writes `env/<date>_env.json` before the runs. It records GPU (model, memory, driver), CPU (i5-14600KF, 14 cores / 20 threads), RAM (95.8 GiB), OS, and Python and library versions per conda env.

OAT's Table 6 used an RTX 4090 and an i9-14900K. Our numbers are never mixed with theirs in one comparison; published numbers appear separately and labelled.

7. Matched-quality protocol (re-tests; completed when target code is available)

Time-to-quality compares the classical trajectory with the ML output under **the target article's own compliance evaluation**. The same thresholding and post-processing are applied to both. The exact procedure is recorded in §9 before any re-test run.

8. Outputs

1. `retest/runs/<run_id>/record.json`: configuration, environment hash, per-iteration arrays, timings, residuals.
2. `power.csv` and `design.npz` (final physical density) in the same run folder.
3. `evidence/baseline_tests.json`: equivalence and unit-test results.

9. Deviations and frozen selections

Date	Item	Value / change	Reason
2026-09-24	S-GPU guard	If PCG cannot reach the true-residual acceptance ($\leq$ 1e-7) within 500 iterations, the solve falls back to a sparse direct solve (scipy SuperLU) on the CPU. The fallback is logged per solve and reported per run	In correctness tests, a synthetic field with 30 % isolated E_min voids made PCG stall at n = 64 (MBB). Realistic fields converged in 10–48 PCG iterations without fallback (`evidence/baseline_tests_gpu.json`). The guard keeps every accepted solve certified; any fallback in a timed run is counted, and its time is included

Date	Item	Value / change	Reason
2026-09-24	Unit-test fields	Solver-accuracy tests use uniform, random-without-voids and a SIMP design at iteration 60 as gating fields. The isolated-void field is kept as a non-gating stress test, with its results recorded	On the isolated-void field, SuperLU and PARDISO also stop at residuals of 1.8e-5 to 3.3e-5 (MBB, n = 64). It measures conditioning, not solver correctness. The pre-registered equivalence requirement (§3) is unchanged and remains gating
2026-09-24	OAT execution environment	New conda env `oat`, cloned offline from `torch5090` (torch 2.11.0+cu128). Added: diffusers 0.40.0, transformers 5.17.0, accelerate 1.15.0, huggingface_hub 1.33.0, datasets 5.0.1, cupy-cuda13x 14.2.0, numba 0.67.0, tabulate 0.10.0, torchvision 0.26.0+cu128, k3d 3.1.1, ipywidgets 8.1.9. Log: `retest/logs/oat_env_install_2026-09-24.log`. Three shims, none editing an OAT file (`retest/oat_harness/shims/`): (1) `sksparse` import-only stub, because scikit-sparse has no Windows wheel; CHOLMOD is never called on the paths used, and OAT's optional CHOLMOD CPU acceleration is therefore not reproduced; (2) the removed alias `cupy.sparse` is restored to `cupyx.scipy.sparse`; (3) torch and CuPy run in separate processes (`OAT_CUPY=1` for pyEDGE)	OAT pins no versions. CUDA 13.0 ships `cusparse64_12.dll` and `cusolver64_12.dll` under the same names as torch's CUDA 12.8 copies, so one process cannot load both (WinError 127). pyEDGE never imports torch, and OAT inference never imports CuPy
2026-09-24	OAT SIMP (pyEDGE) configurations for the same-machine re-test (§7)	The paper does not state which pyEDGE configuration produced Table 6's SIMP-GPU timing. Every configuration that runs on this machine is timed at 150 iterations on 5 seeded NITO_test problems per size, each configuration in its own process. 256^2: **MultiGrid n_level = 5 + OC** (fastest accurate: 20-iteration compliance equals SPSOLVE's) and CG + OC (tol 1e-5, maxiter 1000, fast; 20-iteration compliance departs from SPSOLVE). SPSOLVE is reported from the 20-iteration probe only (14.7–15.0 s/it, so about 37 min per 150-iteration run). 64^2: MultiGrid n_level = 3 + OC, CG + OC, SPSOLVE + OC. The headline comparator is **OAT's fastest accurate configuration**	MultiGrid with the default 3 levels at 256^2 and MultiGrid + PGD crash in cupyx splu → cuSPARSE spSM (cudaErrorIllegalAddress), an environment fault of the CuPy 14 / CUDA 13 stack, not attributed to OAT. The pyEDGE option `coarse_solver='cg'` avoids the crash but runs 19.5 s/it, so it is not used. A `tol` → `rtol` keyword shim makes pyEDGE's CG run under CuPy 14
2026-09-24	Matched-quality procedure (§7), recorded before the time-to-quality run and before any final re-test design is scored	(1) Evaluator: OAT's own compliance evaluation (`scripts/RunComplianceTest.py` logic, in-process in `retest/oat_harness/eval_designs.py`): SPSOLVE, filter radius 1.5, void 1e-6, heavyside off, ground truth = (topology > 0), design variables passed unthresholded, as OAT scores its own outputs. (2) Metric: CE = (c − c_gt)/c_gt × 100 per problem; volume fraction (thresholded at 0.5, and mean) is reported beside it. (3) OAT's design: the output of the last timed sampling run per problem (DDIM, 20 steps, seed 20260924 + 9), without OAT's refinement step, which the harness does not reproduce. (4) Our trajectory: fresh S-GPU runs at budgets K ∈ {5, 10, 15, 20, 30, 40, 50, 75, 100, 150}; time = setup-inclusive wall clock of each fresh run. (5) Matched-quality time = the smallest time among budgets with CE_ours(K) ≤ CE_OAT on that problem; a problem with no such budget is reported as not matched, with no imputed time. (6) Problems: the 5 seeded NITO_test problems per size (256^2: 1015, 1676, 1839, 1935, 1961; 64^2: 15, 676, 839, 935, 961)	Fixes the comparison rule before any CE of the final designs is seen. OAT's reported w/ Ref times enter only rows labelled mixed hardware
2026-09-24	Correction to the row "OAT SIMP (pyEDGE) configurations"	The MultiGrid crash is **not yet attributed**. The earlier wording "an environment fault … not attributed to OAT" stated an untested hypothesis. Final-run observations: MultiGrid(5)+OC at 256^2 and MultiGrid(3)+OC at 64^2 failed on the first problem; every later problem in the same process then failed at once, consistent with a sticky CUDA error, so those later failures are not independent evidence. Attribution follows the `retest/oat_harness/debug_mg.py` run (CUDA_LAUNCH_BLOCKING = 1, one problem per process). Until then, the same-machine OAT-SIMP comparator is CG+OC at both sizes, plus SPSOLVE+OC at 64^2	No accurate MultiGrid timing exists from the final run. The correction keeps an open question from reading as a finding

Date	Item	Value / change	Reason
2026-09-24	Energy protocol (§5), before any energy run of the re-test	The integrated quantity is now `power.draw.instant`, not `power.draw`, and the trapezoid runs over exactly [t_start, t_stop], with power interpolated at the edges; each trace writes `<csv>.window.json`, so every energy can be recomputed from files. `power.draw` stays in the trace as a second column. Energy runs are separate from timing runs (`--power` in `run_ours_on_nito.py` and `run_oat_inference.py`; driver `retest/oat_harness/run_post_retest.sh`); energy per design = window integral / repeats, and our window includes setup of every repeat	nvidia-smi `--help-query-gpu` (driver 616.56) defines `power.draw` on Ampere or newer as a 1 s average, which lags over windows of 1–6 s. The old integrator also padded the window by ±0.2 s. The G2 benchmark energies in `figures/data/baseline_timings.csv` used both old choices and are labelled preliminary; they carry no headline claim
2026-09-24	Matched-quality procedure: evaluator definition and a volume condition (post hoc, after the first scored budgets)	(a) Correction of the evaluator description in the row above: OAT's compliance evaluation calls `MinimumCompliance.FEA(thresshold=True)`, which binarizes the **raw design variables at 0.5, without the density filter**, adds void = 1e-6 and solves (pyEDGE `Problem/CUDA/MinimumCompliance.py`). The filter radius plays no part in the score. (b) Added matching condition: our budget K matches only if CE_ours(K) $\leq$ CE_OAT **and** its thresholded volume fraction $\leq$ max(target, OAT's thresholded volume fraction). (c) Both versions are reported; the version with the volume condition is primary	Seen before the change: problem 1015 at 256^2 scored CE −9.08 % at K = 5, below −8.19 % at K = 150, because thresholding a gray early design can add material. The added condition can only delay or remove a match, so it works against the audit's hypothesis
2026-09-24	CPU placement for all timed runs (§4), before the pinned re-timing	Every timed process runs pinned to the performance cores (logical CPUs 0–11, found with GetSystemCpuSetInformation as EfficiencyClass 1) at HIGH_PRIORITY_CLASS, via `tools/pin.py` (`PIN_PCORES=1`), and the setting is recorded in each run's config. All timed stages are re-run this way (driver `retest/oat_harness/run_pinned_retest.sh`): G2 benchmark (GPU, PARDISO; P1, P2; 64^2, 256^2), OAT SIMP (pyEDGE CG 256^2 and 64^2; SPSOLVE 64^2), ours on NITO (GPU, PARDISO; both sizes), OAT inference (both sizes), time-to-quality runs and energy runs. The pinned run is primary; the earlier default-scheduling runs are reported as a sensitivity check. Designs are compared across runs; if they are identical, the earlier CE scores are reused, otherwise they are re-scored	The i5-14600KF has 6 performance and 8 efficiency cores. Unpinned, OAT inference at 64^2 took 1.02 s at 16:46 UTC, but 0.44 s pinned at 16:48 UTC; it took 0.52 s in the final run. OAT's uncompiled inference launches many small kernels, so its time depends on the core it lands on; our CUDA-graph loop depends less. Pinning every method the same way removes the variation and is expected to favour OAT, so it works against the audit's hypothesis
2026-09-24	MultiGrid crash attribution (closes the open item in the correction row)	A CuPy-only script with no OAT code (`retest/oat_harness/repro_spsm.py`: a 16×16 Q4 stiffness with a random SIMP-like field, re-factored with `cupyx.scipy.sparse.linalg.splu` and solved 60 times per factorization) fails at factorization 6 with the same error on the same call path: `splu(...).solve` → `cusparse.spsm` → `spSM_solve` → cudaErrorIllegalAddress (`retest/logs/repro_spsm_20260924T172745Z.txt`; CuPy 14.2.0, CUDA runtime 13020, RTX 5090, driver 616.56). pyEDGE's own runs fail the same way at OC iteration 35 (256^2, 5 levels) and 15 (64^2, 3 levels) (`retest/logs/debug_mg_*.txt`). The crash is therefore attributed to the CuPy/cuSPARSE stack on this machine, not to OAT's code. OAT's MultiGrid path cannot be timed here; the same-machine OAT-SIMP comparator stays pyEDGE CG+OC (both sizes) and SPSOLVE+OC (64^2). The 20-iteration MultiGrid(5) probe (0.2465 s per iteration at 256^2, unpinned) is reported only as an indication	Evidence replaces the earlier hypothesis. Changing the CuPy or CUDA version to work around it was not attempted
2026-09-24	Per-design comparisons with OAT include our setup (post hoc; found while checking the 64^2 CPU time-to-quality run)	§4 excludes one-time setup from the timed loop, and the G2 gate keeps that definition. But OAT's inference time covers all of its per-design work except the data collator and model loading. So every **per-design comparison with OAT** now uses our setup-inclusive wall time: filter and assembly structures, the multigrid hierarchy or PARDISO's symbolic phase, and the loop. It comes from the fresh 150-iteration runs of the time-to-quality series. PARDISO is added to that series at 256^2. Loop-only times stay reported as the pre-registered benchmark figure. pyEDGE's setup stays excluded, which favours OAT's classical code	Measured setup per design: PARDISO 0.81–0.86 s at 64^2 (its 150-iteration loop takes about 0.6 s); GPU 0.09–0.20 s. Excluding it made our baseline look faster than it is at 64^2; including it works against the audit's hypothesis

Date	Item	Value / change	Reason
2026-09-24	HGTO re-test (P00512; §7), recorded before any HGTO or HGTO SIMP–OC run on this machine	(1) **Target code:** github.com/Liukz233/HGTO at commit d9828ef06ed6a75329ecb97158271e94bbaa08f2, run unmodified in conda env `hgto` (a clone of `torch5090` plus `pip install -e HGTO[fast-cpu,test]`; log `retest/logs/hgto_env_install_2026-09-24.log`). (2) **Problems:** the six `configs/linear2d/*.yaml` files, unchanged: cantilever 120×40, 240×80, 480×160 and 960×320; half-MBB 120×40; inclined load 120×40. (3) **Methods:** (a) HGTO as released, on the GPU (`hgto run --config C`, default `--threads 2`); (b) the authors' SIMP–OC as released (`--method oc`, pypardiso-spd), with `--threads 2` (the article's setting) and `--threads 6` (one thread per physical performance core); (c) our GPU port of the same SIMP–OC (`retest/hgto_harness/oc_gpu.py`). The port keeps the filter, tanh projection, ρ_min floor, E_min = 1e-6, penalty and β continuation, stage budgets, OC update with its bisection, volume restoration and physical stopping rule. It moves only the state solve and the array arithmetic to the GPU, using the S-GPU multigrid PCG of §3. Its acceptance is tightened to a true residual $\leq$ 1e-8 (recurrence 1e-9), because the target's stopping rule requires an equilibrium residual $\leq$ 1e-8. (4) **Equivalence gate before any timing:** on the three 120×40 configurations, the per-update compliance of (c) matches (b, 2 threads) within 1e-5 relative over the first 50 updates. On all six configurations, (c)'s final compliance from the common independent evaluator lies within 1 % of (b)'s, with the same termination status. A failure stops the re-test for diagnosis. (5) **Timing:** every run in its own process, pinned (`tools/pin.py`, `PIN_PCORES=1`). The timed interval is the target's own `wall_s` scope: case construction, setup, optimization and the final independent scikit-fem evaluation, with imports and file output excluded. (c) is timed over the same scope. One discarded warm-up and three timed repeats per method and configuration, reported as the median. A method–configuration pair whose first run exceeds 300 s gets that single run only, without warm-up. (6) **Quality:** every final design is scored by the target's own independent evaluator (`hgto evaluate`: scikit-fem, p = 3, E_min = 1e-6). (7) **Primary comparison:** converged-to-converged wall time under the target's own stopping rule, as in the article's Table 1 and Table S3, with each design's compliance reported beside it. **Secondary:** matched quality. This is the elapsed time at the first final-stage update of (c) whose compliance is at most HGTO's independently evaluated final compliance with volume error $\leq$ 1e-7, plus the final-evaluation time. (8) The article's own times (RTX 6000 Ada; Threadripper PRO 5995WX, two threads) are reported separately and labelled	The article's largest speed claims (about 4× at 480×160 and about 10× at 960×320) compare GPU HGTO with two-thread CPU SIMP–OC. The re-test separates the choice of hardware and solver from the method and holds the formulation fixed, as §1 requires
2026-09-24	HGTO execution environment (before any timed HGTO run)	In env `hgto`, torch's bundled `libiomp5md.dll` is renamed so that torch and the MKL used by pypardiso share one Intel OpenMP runtime (`<env>/Library/bin/libiomp5md.dll`). No HGTO file is changed, and `KMP_DUPLICATE_LIB_OK` is not used. Log: `retest/logs/hgto_env_install_2026-09-24.log`. Untimed smoke runs on `cantilever_120x40` after the fix: HGTO's SIMP–OC (two threads) gives compliance 175.08232935120 in 298 updates, converged; the article's `benchmarks/reference/linear.csv` lists 175.08232935138. The GPU port gives 175.08232934292 in 298 updates, converged	Without the fix, every HGTO run that calls PARDISO after importing torch aborts with "OMP: Error #15". That covers HGTO's own SIMP–OC and the final independent evaluation of every run

Date	Item	Value / change	Reason
2026-09-24	P00150 re-test (Halle et al. 2021; §7), recorded before any P00150 run	(1) **Target:** the article's trained predictor `model_predictor_64x64` and its 100 validation cases with the stored top88 results (Mendeley data, CC BY 4.0; `retest/targets/P00150_mendeley/`, SHA-256 in `DOWNLOAD_MANIFEST.md`), used unmodified. The predictor runs in a new conda env `tf_p00150` (Python 3.11, TensorFlow 2.15, CPU only, because TensorFlow after 2.10 has no GPU support on native Windows). (2) **Problems:** each validation input decodes to a 64×64 problem: left edge clamped, one unit point load at a node of the 8×8 input grid, and a target volume fraction (`retest/p00150_harness/problems.py`). Settings follow the article's Table 2: p = 3, r_min = 3, $\nu = 0.3$. Compliance is computed with E = 1 and a unit load, then multiplied by $100^2/195{,}000$ to give the article's units. (3) **Reconstruction check before any comparison:** our top88 port (S-SuperLU path) must reproduce the stored top88 compliance within 1 % at top88's stopping point (design change ≤ 0.01) on at least 95 of the 100 cases. Otherwise the re-test stops for diagnosis. (4) **Methods:** (a) the predictor: batch size 1, a compiled `tf.function` call per case, pinned, timed as the median of 5 calls after 20 warm-up calls; (b) the top88 formulation with S-SuperLU (the typical Python port), S-PARDISO and S-GPU, each run to top88's stopping rule and timed setup-inclusive, with one warm-up and three repeats per case, reported as the median. (5) **Quality:** one evaluator scores every design: fem2d, E_min = 1e-9, p = 3, SuperLU. It applies to the predictor's continuous output (as the article evaluates it), the stored top88 designs and our designs. (6) **Primary:** the ratio of mean per-design times over the 100 cases (the article's statistic), comparing the strong baseline run to its stopping rule with the predictor. **Secondary:** matched quality, meaning per case the setup-inclusive time to the first iteration whose compliance is at most the predictor's; the OC update holds the volume exactly. A case that never matches is reported as not matched and ranked slowest. Also secondary: the typical Python port on our machine against the predictor, which repeats the article's comparison. (7) The article's values (1.9 s and 7.3 ms, 259×; the stored per-case top88 times average 1.896 s) are reported separately and labelled	The article's 259× compares a trained predictor with top88 on unstated hardware. The re-test puts both on one machine and adds the strong baseline
2026-09-24	P00150 execution environment (before any P00150 predictor run; amends item 1 of the P00150 row)	The predictor runs in a separate Python virtual environment, created from the interpreter of conda env `Pytorch` (Python 3.10.18), with `tensorflow==2.15.1` from PyPI (CPU only on native Windows). The `Pytorch` env is not modified. Log: `retest/logs/tf_p00150_env_install_2026-09-24.log`	Creating a new conda env could require accepting the Anaconda channel terms; a venv from an existing interpreter avoids that. TensorFlow 2.15 still loads SavedModels through `tf.keras`, as the harness expects
2026-09-24	P00150 reconstruction diagnosis (the pre-registered check stopped the re-test; recorded before the check is re-run)	Under the first mapping (v1: the whole left edge clamped; the input's y load taken as fem2d's +y), the untimed SuperLU scan gave compliance ratios (stored ÷ reproduced) of 0.55–1.49 over the first 11 cases, so the check failed and the re-test stopped. Diagnosis evaluated the article's own stored top88 designs with our finite-element model under candidate mappings (`retest/runs/p00150/_diagnosis/`). Two errors in our reconstruction were found. (1) The y load component follows a y-down frame, so it is negated in fem2d; this left ratios of 0.99–1.37 with a spread of ±8 %. (2) Only the 9 fine nodes that coincide with the clamped coarse nodes are supports, not the whole edge. With both fixes (mapping v2), 60 of 100 stored designs evaluate within 1 % of their stored compliance (many to 4 digits), and the other 40 evaluate 1–12 % above it. The misfits are bending-dominated, lower-volume cases, not yet explained; plane strain, a filtered field and E_min = 1e-3 were tested and make the fit worse or leave it unchanged. The pre-registered check (item 3: our top88 run to its stopping point against the stored compliance, $\geq$ 95 of 100 within 1 %) is now re-run under mapping v2. If it fails again, the P00150 speed comparison is not reported as a re-test; only the diagnosis is reported	The check did its job: the first reconstruction was wrong. Both fixes rest on the article's own data, the stored designs and compliances, not on any speed result

Date	Item	Value / change	Reason
2026-09-25	P00150 reconstruction check, result (mapping v2)	Failed. Our top88 port matched the stored top88 compliance within 1 % at top88's stopping point on 57 of 100 cases (SuperLU port; the GPU solver gives the same 57, case for case); the pass condition was at least 95. In the other cases the stored compliance lies up to 19 % below our reproduction, mostly in bending-dominated cases. Per the diagnosis row, no P00150 speed comparison is reported; the manuscript reports the check and the diagnosis only (`evidence/p00150_check.json`, `retest/runs/p00150/_diagnosis/`). The predictor was never run, and the TensorFlow venv was removed as unused	Pre-registered stopping rule applied as written
2026-09-25	OAT re-test, post hoc extension (after the primary re-test was reported)	20 further NITO_test problems per size, drawn with `random.Random(20260925)` from the problems of that size excluding the primary five, and 5 OAT sampling seeds (20261924 + 1000·k) for all 25 problems. Same harness functions, same budgets, same evaluator and both matching rules; every timed step pinned (`retest/oat_harness/extended.py`, `run_extended.sh`, log `retest/logs/oat_extended_2026-09-25.log`, summary `analysis/oat_extended.py` → `evidence/oat_extended.json`). Per-problem ratio = median over the 5 samples; unmatched samples right-censored; exact two-sided sign test across problems. The pre-registered five problems remain the primary result	Five problems and one OAT sample per problem cannot show how stable the matched-quality ratio is; the extension is secondary and labelled post hoc
2026-09-25	Strong baseline, OC update (bug fix found by the extension)	On NITO_test problem 1974 (256^2) a filtered sensitivity of +3.2e-16 made `sqrt(-dc/dv/lmid)` NaN at iteration 3, so the bisection never converged (GPU: RuntimeError; CPU: division by zero). Sensitivities are now clamped at 0 in `simp_gpu.py` and `simp_cpu.py`. The clamp is a no-op whenever all sensitivities are ≤ 0, which holds for every earlier run (any positive value would have stopped that run); re-running problem 1015 at 256^2 reproduced its recorded compliance to a relative 2.2e-13	Rounding-level robustness fix; no earlier result changes

3. Screening and full-text reason codes

Title/abstract screening codes (plan/PROTOCOL.md §3–4)

Each pool record gets exactly one decision and one reason code.

Decision	Code	Meaning
include	I-SPEED	ML for TO or architected-material inverse design, and the abstract claims a speed or cost advantage over a classical, non-learned optimization method. Examples: faster, real-time, accelerate, reduces computational cost, without iterative optimization, fewer FE evaluations
include	I-QUALITY	Same scope, but the abstract claims only better design quality than a classical method (secondary stratum Q)
include	I-UNCLEAR	Same scope; a comparison with a classical method is plausible but not stated in the abstract. The full text decides
exclude	E-SCOPE	Not TO or architected-material inverse design, e.g. network "topology", topological data analysis, photonics
exclude	E-DOMAIN	TO in an excluded physics domain: fluid flow, acoustics, photonics, electromagnetics
exclude	E-NOML	No learned model; classical TO only
exclude	E-TYPE	Review, survey, thesis, dataset-only or benchmark-only paper with no method claim, tutorial, perspective
exclude	E-FORWARD	ML used only for forward analysis or prediction; no optimization-level comparison
exclude	E-MLONLY	Compares only with other ML methods; no classical comparator mentioned. Tallied for C4
exclude	E-NOCLAIM	Proposes an ML-for-TO method but makes no comparative claim of any kind
exclude	E-OTHER	Anything else; explain in the rationale

Extra fields per record:

1. method family: convolutional neural network, generative adversarial network, variational autoencoder, diffusion or flow model, transformer, graph neural network, language-model agent, reinforcement learning, neural reparameterization, surrogate using a multilayer perceptron or Gaussian process, or other.

2. physical objective: compliance, stress, thermal, compliant mechanism, multiphysics, architected-material homogenization, or other.

3. dimension: two-dimensional, three-dimensional, both or unclear.

4. rationale: one sentence, ≤ 30 words, in the screener's own words (no copying from the abstract).

When unsure, prefer I-UNCLEAR over an exclude. The full text is the arbiter, and exclusions at this stage are cheap to recover later.

Full-text exclusion codes (coding stage; plan/PROTOCOL.md §11, clarification of 2026-09-24)

An article excluded at full text gets exactly one code from this closed list. When several apply, use the first one in the table order. Put any explanation in notes, not in the exclusion-reason field.

Order	Code	Meaning
1	FT-DUPLICATE	Another version of an article already in the pool. Code the primary version (version of record if open access, else the latest arXiv version) and name it in notes
2	FT-SCOPE	Not TO or architected-material inverse design
3	FT-TYPE	Review, survey, thesis, dataset-only or benchmark-only paper with no method claim, tutorial, perspective
4	FT-DOMAIN	TO in an excluded physics domain: fluid flow, acoustics, photonics, electromagnetics
5	FT-NOML	No learned model; classical TO only
6	FT-FORWARD	ML used only for forward analysis or prediction; no optimization-level comparison
7	FT-MLONLY	Compares only with other ML methods; no classical comparator is run, timed or cited. Tallied for C4
8	FT-NOBASELINE	No comparison of any kind with a classical method: the article does not run, time or cite a classical method's cost or result. A qualitative motivation such as "TO is slow" is not a comparison
9	FT-OTHER	Anything else; explain in notes

First-pass codings written before this list used free-text labels. `analysis/codes.py` maps them without editing the frozen first-pass files: FT-ML-only → FT-MLONLY; FT-NOCOMPARISON → FT-NOBASELINE. Any other label stops the analysis with an error, so the record must be recoded.

4. Coding instructions

Quoted instrument (machine-readable labels retained).

> **Full-text coding instructions (first AI pass; a blind second AI pass and AI adjudication follow)**
>
> Apply `plan/PROTOCOL.md` §3–6 to one included article. Fill one record that follows `audit/schema/coding_schema.json`, with `stage = "llm_prefill"` and `coder.human_initials = null`.
>
> *Steps*
>
> 1. **Access.** Read the full text: arXiv HTML (`https://arxiv.org/html/<id>`) or the PDF (`https://arxiv.org/pdf/<id>`), or the version of record if it is open access.
>   (a) Record `access` and `citation.version_coded`.
>   (b) If only the abstract is reachable, set `access = "abstract_only"` and do not assign rule verdicts; use `verdict = "na"` with evidence "no full text".
> 2. **Full-text inclusion.** Confirm §3 and §4. If the article fails them, set `included.decision = "exclude"` with `reason = FT-<short reason>`, and stop after the citation and the reason.
> 3. **Primary outcome** (§5.1), in this order:

(a) the first quantitative comparison with a **classical** method in the abstract;
(b) otherwise in the conclusion, then the introduction, then the main text;
(c) otherwise the first qualitative comparison, in the order abstract → conclusion → introduction.
(d) Write `claim.primary_outcome_paraphrase` in your own words.
(e) Give its `location`. Give at most one verbatim `key_phrase` (< 15 words).
(f) Record other candidates in `claim.alternatives`.
(g) Set `included.stratum`: `speed` if the primary outcome compares speed, cost or a proxy for them; `quality` if it compares design quality only.

4. **Baseline facts.** Fill every `baseline` field the article states. Write "not stated" when absent. Never guess.
   (a) Report numbers exactly as printed, with units.
   (b) If a speed-up factor is stated, or can be derived as classical time ÷ ML time from printed numbers, put it in `claimed_speedup` and say how it was derived in `claimed_speedup_basis`.
5. **Rules R1–R4** (§5.3). Set a verdict for each and name the protocol `marker` that applies, e.g. `R2(c)`, `R1-unmatched`, `R3-no-data-cost`, `R4-image-metric`.
   (a) `evidence` = locator + paraphrase.
   (b) **Unsure means pass** (template convention). Use `partial` only for the partial cases the protocol lists.
   (c) R2 markers (a) and (e) need our re-test or a per-iteration time. Without them, use (b), (c) or (d) only when the article's own text establishes them. Otherwise R2 is `pass` or `partial`.
6. **Reporting indicators** (§6). Fill every field.
7. **retest_candidate.** true only if public code exists, the article's benchmark can be run, and nothing forbids running it.

Style and limits

1. Paraphrase. At most one verbatim phrase per article, under 15 words.
2. No inferences stated as facts. Mark inferences "[inferred]" inside the evidence text.
3. Numbers must be traceable: every numeric field gets a locator.
4. Output valid JSON only, one file per article: `audit/coding/llm_prefill/<pid>.json`.

5. Adjudication instructions

Quoted instrument (machine-readable labels retained).

Adjudication instructions (AI verification; plan/PROTOCOL.md §11, 2026-09-24)

An adjudicator settles every item on which the two independent codings of one article disagree. The inputs are:

1. the first coding, `audit/coding/llm_prefill/<pid>.json`;
2. the blind second coding by a different model, `audit/coding/second_pass/<pid>.json`;
3. the list of disagreeing items, `audit/coding/adjudication_queue.json`, written by `analysis/compare_passes.py`.

The compared items are: access, inclusion decision, full-text exclusion code, stratum, R1–R4 verdicts, and the claimed speed-up (the two values agree when both are absent, or when they are within 10 % relative).

Procedure

1. **Read the article yourself.** Follow the access rules below. Neither coding is the default. Decide each disputed item from the article text, `plan/PROTOCOL.md` §3–6 and §11, `audit/schema/coding_instructions.md` and `audit/schema/screening_codes.md`.
2. **Access disputes.** If either coder reached the full text, try the same legal route. If you reach the full text, the final record is a full-text coding. Start from the coding that used the full text, and settle its other items as usual. If no legal full text is reachable, the final record has `access = "abstract_only"` (or `"no_access"`) and all rule verdicts `na`.
3. **Build the final record.**
   (a) Start from the coding that you judge closer to the article.
   (b) Set every disputed item to your decision.
   (c) Update the locator and evidence text of every field you change, marking inferences "[inferred]".
   (d) Keep the other fields unless they contradict your decisions. For example, an exclusion needs stratum `none` and all verdicts `na`.
4. **Record the adjudication.** Add a top-level `adjudication` object with these fields:
   (a) `adjudicator_model` and `date`;
   (b) `based_on`: `"first"` or `"second"`;
   (c) `items`: one entry per disputed item. Each entry holds `first` (value), `second` (value), `decision` (value) and `reason` (one or two sentences, with a locator).
5. **Write** `audit/coding/records/<pid>.json` with `stage = "ai_adjudicated"` (Write tool, valid JSON). Never change the two input codings.
6. **Validate** with `python -m audit.coding.validate_records records <pid> ...`, run from the project root with base Anaconda, which has jsonschema.

Rules that decide frequent disputes

1. **No full text, no full-text exclusion.** An article whose full text no coder could read legally keeps its screening decision: include, reason "I-UNCLEAR (no accessible full text)", stratum `none`, all verdicts `na`.
2. **Unsure means pass** for R1–R4, which is the template convention. Use `partial` only for the partial cases that §5.3 lists.
3. **Primary outcome** (§5.1): the first quantitative comparison with a classical method, in this order: abstract, then conclusion, then introduction, then main text. Otherwise, the first qualitative comparison. The stratum follows the primary outcome: `speed` if it compares speed, cost or a proxy for them; `quality` if it compares design quality only.
4. **Full-text exclusions** use the closed list in `screening_codes.md`. When several apply, use the first in table order.
5. **FT-DUPLICATE** marks the non-primary version. The primary version is the version of record if it is open access; otherwise it is the latest arXiv version. Name the other pool id in `notes`.
6. **Stratum Q (quality-only primary outcome)** (PROTOCOL §11, 2026-09-24): R3 is `na`; R2 is `na` unless criterion (b) applies (the abstract reports a weaker baseline than the results or appendix); R1 and R4 apply as written.

Access rules

1. Use legal copies only: arXiv, the publisher's open-access version (including its own file host), institutional or funder repositories, and author pages.
2. Never bypass a CAPTCHA, a bot check, a login or a paywall.
3. Send no personal data: use a generic User-Agent, and no email, `mailto`, Unpaywall, API keys or logins.
4. Keep downloads in a scratch folder outside the project, and delete them afterwards.

Duplicate tie-break (PROTOCOL §11, 2026-09-24)

When two pool records hold the same version of one article, the record earlier in the seeded order (`audit/screening/pilot_final_order.csv`; a record outside the order counts as later) carries the coding, and the other is FT-DUPLICATE. Records of different versions keep the primary-version rule.

6. Search queries (frozen)

Quoted instrument (machine-readable labels retained).


```
{
  "frozen_utc":  "2026-09-24",
  "search_end_date":  "2026-09-24",
  "protocol":  "plan/PROTOCOL.md v1.0 section 2",
  "arxiv":  [
    {
      "id":  "AX1",
      "query":  "(abs:\"topology optimization\" OR abs:\"topology optimisation\" OR ti:\"topology optimization\" OR ti:\"topology optimisation\") AND (abs:\"deep learning\" OR abs:\"neural network\" OR abs:\"neural networks\" OR abs:\"machine learning\")"
    },
    {
      "id":  "AX2",
      "query":  "(abs:\"topology optimization\" OR abs:\"topology optimisation\") AND (abs:diffusion OR abs:GAN OR abs:\"generative adversarial\" OR abs:\"variational autoencoder\" OR abs:\"flow matching\" OR abs:generative)"
    },
    {
      "id":  "AX3",
      "query":  "(abs:\"topology optimization\" OR abs:\"topology optimisation\") AND (abs:\"large language model\" OR abs:\"large language models\" OR abs:LLM OR abs:LLMs OR abs:agent OR abs:agents)"
    },
    {
      "id":  "AX4",
      "query":  "(abs:\"topology optimization\" OR abs:\"topology optimisation\") AND abs:\"reinforcement learning\""
    },
    {
      "id":  "AX5",
      "query":  "(abs:\"topology optimization\" OR abs:\"topology optimisation\") AND (abs:\"physics-informed\" OR abs:\"implicit neural\" OR abs:reparameterization OR abs:reparametrization OR abs:\"neural field\" OR abs:\"neural fields\")"
    },
    {
      "id":  "AX6",
      "query":  "(abs:metamaterial OR abs:metamaterials OR abs:microstructure OR abs:\"architected material\" OR abs:\"architected materials\" OR abs:\"cellular material\" OR abs:lattice) AND abs:\"inverse design\" AND (abs:\"deep learning\" OR abs:\"neural network\" OR abs:\"machine learning\") AND (abs:\"topology optimization\" OR abs:homogenization)"
    }
  ],
  "openalex":  [
    {
      "id":  "OA1",
      "filter":  "title_and_abstract.search:(\"topology optimization\" OR \"topology optimisation\") AND (\"deep learning\" OR \"neural network\" OR \"neural networks\" OR \"machine learning\"),from_publication_date:2017-01-01"
```

```
    },
    {
      "id": "OA2",
      "filter": "title_and_abstract.search:(\"topology optimization\" OR \"topology
optimisation\") AND (\"generative adversarial\" OR \"diffusion model\" OR \"variational
autoencoder\" OR \"generative model\" OR \"flow matching\"),from_publication_date:2017-01-01"
    },
    {
      "id": "OA3",
      "filter": "title_and_abstract.search:(\"topology optimization\" OR \"topology
optimisation\") AND (\"large language model\" OR \"reinforcement learning\" OR \"physics-informed\"
OR \"implicit neural\"),from_publication_date:2017-01-01"
    }
  ],
  "forward_citation_anchors": [
    {
      "id": "FC1",
      "title": "Neural networks for topology optimization",
      "hint": "Sosnovik & Oseledets, arXiv 1709.09578; DOI added 2026-09-24 (v1.1) after the title
fallback resolved a wrong work",
      "doi": "10.1515/rnam-2019-0018"
    },
    {
      "id": "FC2",
      "title": "3D topology optimization using convolutional neural networks",
      "hint": "Banga et al., arXiv 1808.07440"
    },
    {
      "id": "FC3",
      "doi": "10.1115/1.4049533",
      "hint": "Nie et al. 2021 TopologyGAN, J. Mech. Des."
    },
    {
      "id": "FC4",
      "title": "TOuNN: Topology Optimization using Neural Networks",
      "hint": "Chandrasekhar & Suresh 2021, SMO"
    },
    {
      "id": "FC5",
      "doi": "10.1007/s00158-022-03347-1",
      "hint": "Woldseth et al. 2022, SMO"
    },
    {
      "id": "FC6",
      "title": "Diffusion Models Beat GANs on Topology Optimization",
      "hint": "Maze & Ahmed, arXiv 2208.09591"
    }
  ],
  "backward_citation_sources": [
    {
      "id": "BC1",
      "doi": "10.1007/s00158-022-03347-1",
      "hint": "Woldseth et al. 2022 reference list via OpenAlex referenced_works"
    },
    {
```

```
      "id": "BC2",
      "doi": "10.1007/s00158-025-04135-3",
      "hint": "Sanu et al. 2025 reference list via OpenAlex referenced_works"
    },
    {
      "id": "BC3",
      "arxiv": "2510.23667",
      "hint": "OAT reference list, from the arXiv HTML bibliography (OpenAlex W7197034560 has no
referenced_works); titles matched to the pool, as PROTOCOL §2.3 specifies; added in v1.2 because
v1.0 omitted it"
    }
  ],
  "version": "1.2"
}
```


7. Coding schema

Quoted instrument (machine-readable labels retained).


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Full-text coding record (plan/PROTOCOL.md §5-7)",
  "type": "object",
  "required": [
    "pid",
    "citation",
    "access",
    "coder",
    "included",
    "claim",
    "rules",
    "reporting",
    "stage"
  ],
  "properties": {
    "pid": {
      "type": "string",
      "description": "pool id, e.g. P00042"
    },
    "citation": {
      "type": "object",
      "required": [
        "title",
        "year"
      ],
      "properties": {
        "title": {
          "type": "string"
        },
        "authors": {
          "type": "string"
        },
        "year": {
          "type": "integer"
```

```
        },
        "venue": {
          "type": "string"
        },
        "doi": {
          "type": "string"
        },
        "arxiv_id": {
          "type": "string"
        },
        "version_coded": {
          "type": "string",
          "description": "e.g. arXiv v2, or version of record"
        }
      }
    },
    "access": {
      "type": "string",
      "enum": [
        "full_text",
        "abstract_only",
        "no_access"
      ]
    },
    "coder": {
      "type": "object",
      "properties": {
        "llm_model": {
          "type": "string"
        },
        "llm_date": {
          "type": "string"
        },
        "human_initials": {
          "type": [
            "string",
            "null"
          ],
          "description": "legacy field, null: verification is by a blind second AI pass and AI
adjudication (PROTOCOL §11, 2026-09-24)"
        },
        "human_date": {
          "type": [
            "string",
            "null"
          ]
        }
      }
    },
    "stage": {
      "type": "string",
      "enum": [
        "llm_prefill",
        "ai_second_pass",
```

```
        "ai_agreed",
        "ai_adjudicated",
        "human_verified"
      ],
      "description": "llm_prefill = first AI pass; ai_second_pass = blind second AI pass; ai_
agreed / ai_adjudicated = final records (PROTOCOL §11, 2026-09-24); human_verified = legacy,
unused"
    },
    "included": {
      "type": "object",
      "required": [
        "decision",
        "reason"
      ],
      "properties": {
        "decision": {
          "type": "string",
          "enum": [
            "include",
            "exclude"
          ]
        },
        "reason": {
          "type": "string",
          "description": "screening code from screening_codes.md, or FT-<reason> for full-text
exclusions"
        },
        "stratum": {
          "type": "string",
          "enum": [
            "speed",
            "quality",
            "none"
          ],
          "description": "speed = primary outcome compares speed/cost (headline C1); quality =
quality-only (stratum Q)"
        },
        "method_family": {
          "type": "string"
        },
        "physics": {
          "type": "string"
        },
        "dim": {
          "type": "string"
        }
      }
    },
    "claim": {
      "type": "object",
      "properties": {
        "primary_outcome_paraphrase": {
          "type": "string",
          "description": "coder's own words, <= 40 words"
        },
```

```
        "key_phrase": {
          "type": "string",
          "description": "at most one verbatim phrase, < 15 words"
        },
        "location": {
          "type": "string",
          "description": "abstract | conclusion | introduction | section/table/figure id"
        },
        "quantitative": {
          "type": "boolean"
        },
        "claimed_speedup": {
          "type": [
            "number",
            "null"
          ],
          "description": "numeric factor if stated or derivable (classical time / ML time)"
        },
        "claimed_speedup_basis": {
          "type": "string"
        },
        "direction": {
          "type": "string",
          "enum": [
            "outperform",
            "underperform",
            "similar_or_mixed"
          ]
        },
        "alternatives": {
          "type": "string",
          "description": "other candidate primary outcomes, briefly"
        }
      }
    },
    "baseline": {
      "type": "object",
      "properties": {
        "classical_method": {
          "type": "string",
          "description": "e.g. SIMP with OC, MMA, BESO, level set, homogenization-based inverse
design"
        },
        "implementation": {
          "type": "string",
          "description": "e.g. MATLAB 88-line, Python port, commercial (Abaqus/ANSYS/Tosca), GPU
code, unknown"
        },
        "solver": {
          "type": "string",
          "description": "direct / iterative / multigrid / unknown"
        },
        "hardware_classical": {
          "type": "string"
```

```
    },
    "hardware_ml": {
      "type": "string"
    },
    "problem_size": {
      "type": "string",
      "description": "e.g. 64x64 elements, 3D 64^3"
    },
    "classical_iterations": {
      "type": "string",
      "description": "fixed count or stopping rule"
    },
    "classical_time": {
      "type": "string"
    },
    "ml_time": {
      "type": "string"
    },
    "training_time": {
      "type": "string"
    },
    "data_generation_cost": {
      "type": "string"
    },
    "quality_metric": {
      "type": "string",
      "description": "compliance error, volume error, MSE/IoU, surrogate-predicted, ..."
    },
    "quality_gap": {
      "type": "string",
      "description": "reported difference between ML output and classical result"
    }
  }
},
"rules": {
  "type": "object",
  "description": "verdict in {pass, partial, fail, na}; evidence = locator + paraphrase",
  "properties": {
    "R1": {
      "$ref": "#/$defs/verdict"
    },
    "R2": {
      "$ref": "#/$defs/verdict"
    },
    "R3": {
      "$ref": "#/$defs/verdict"
    },
    "R4": {
      "$ref": "#/$defs/verdict"
    }
  }
},
"reporting": {
  "type": "object",
```

```
"properties": {
  "n_test_cases": {
    "type": "string"
  },
  "ood_test": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "unclear"
    ]
  },
  "seeds_or_variance": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "unclear"
    ]
  },
  "failure_rate_reported": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "unclear"
    ]
  },
  "timing_protocol_stated": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "partial"
    ]
  },
  "runtimes_all_cases": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "selective",
      "na"
    ]
  },
  "code_available": {
    "type": "string",
    "enum": [
      "yes",
      "no",
      "on_request"
    ]
  },
  "data_available": {
```

```
          "type": "string",
          "enum": [
            "yes",
            "no",
            "on_request"
          ]
        },
        "hardware_per_method_stated": {
          "type": "string",
          "enum": [
            "yes",
            "no",
            "partial"
          ]
        }
      }
    },
    "retest_candidate": {
      "type": "boolean",
      "description": "public code + runnable + licence allows running"
    },
    "notes": {
      "type": "string"
    }
  },
  "$defs": {
    "verdict": {
      "type": "object",
      "required": [
        "verdict",
        "evidence"
      ],
      "properties": {
        "verdict": {
          "type": "string",
          "enum": [
            "pass",
            "partial",
            "fail",
            "na"
          ]
        },
        "marker": {
          "type": "string",
          "description": "which protocol marker applied, e.g. R2(c) or R1-unmatched"
        },
        "evidence": {
          "type": "string",
          "description": "locator + paraphrase; no long quotes"
        }
      }
    }
  }
}
```

8. Masked-protocol note

The blind second coder received a copy of the protocol in which every record identifier was replaced by `P#####` (12 occurrences), so that the protocol's worked examples and deviation entries could not reveal the first coding of any record. The copy was otherwise identical to the protocol in force when the second pass began. It is released with the coding records (`PROTOCOL_pids_masked.md`).